

SAT-Based Free-Topology Transistor Network Synthesis under Stack-Height, Multi-Stage and Multi-Output Options

Byeonggon Kang
UC San Diego
San Diego, USA
b8kang@ucsd.edu

Sehyeon Kim
POSTECH
Pohang, Republic of Korea
sehyeon2001@postech.ac.kr

Seokhyeong Kang
POSTECH
Pohang, Republic of Korea
shkang@postech.ac.kr

Alan Mishchenko
UC Berkeley
Berkeley, USA
alanmi@berkeley.edu

Masahiro Fujita
National Institute of
Advanced Industrial
Science and Technology
Tokyo, Japan
fujita@ee.t.u-tokyo.ac.jp

Susmita Sur-Kolay*
Indian Institute of
Engineering Science and
Technology, Shibpur
Howrah, India
ssk@isical.ac.in

Bill Lin
UC San Diego
San Diego, USA
billin@eng.ucsd.edu

Chung-Kuan Cheng
UC San Diego
San Diego, USA
ckcheng@ucsd.edu

Abstract

Transistor-network synthesis is important for generating compact circuit implementations, and flexible synthesis options can directly affect standard-cell quality. We propose a SAT-based exact synthesis framework for *free-topology* transistor networks that allows internal nets generated in one stage to be reused only by subnetworks in subsequent stages under an acyclic constraint. The framework jointly supports stack-height-constrained multi-stage synthesis and multi-output synthesis, together with design-space pruning techniques including structural constraints, symmetry breaking, and tighter internal-net bounds. Experiments validate the framework on 3-input P-class functions, the 53-function NSP benchmark, and multi-output functions. Under a 4-stack constraint, the proposed method reduces transistor count by 21% over MiniTNtk [22] on 3-input P-class functions, matches the known optimum on the NSP benchmark [9] under a relaxed stack-height bound, and improves or matches the best reported counts on selected multi-output benchmarks. Circuit-level simulations also show improved delay-power-area (DPA) compared with prior 3-input implementations, with best improvements of 64.8% over MiniTNtk [22] and 50.9% over L2L [14]. Under a stricter 2-stack setting, the framework still covers all 3-input P-class functions, trading 10.7% transistor overhead for improved rise/fall balance. Design-space pruning reduces normalized runtime by 53.6–97.7% across the evaluated cases.

CCS Concepts

• **Hardware** → **Transistor-level synthesis**; *Circuit optimization*; Standard cell libraries.

Keywords

Standard cell circuits, transistor network synthesis, topology, SAT

ACM Reference Format:

Byeonggon Kang, Sehyeon Kim, Seokhyeong Kang, Alan Mishchenko, Masahiro Fujita, Susmita Sur-Kolay, Bill Lin, and Chung-Kuan Cheng.

*This work was conducted during her visit to UC San Diego.



This work is licensed under a Creative Commons Attribution 4.0 International License. ICCAD '26, San Jose, CA, USA

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2873-0/2026/11
<https://doi.org/10.1145/3831252.3834143>

2026. SAT-Based Free-Topology Transistor Network Synthesis under Stack-Height, Multi-Stage and Multi-Output Options. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD '26)*, November 08–12, 2026, San Jose, CA, USA. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3831252.3834143>

1 Introduction

Transistor-network synthesis enables compact and functionally diverse circuit implementations. As layout pitch and interconnect scaling continue to slow [4], chip-level power, performance, and area (PPA) gains increasingly rely on circuit- and standard-cell-level optimization [20]. In this regime, transistor-network synthesis plays an important role by enabling PPA-efficient circuit implementations and a broader set of logic functions in standard-cell libraries.

Prior transistor-network synthesis has largely been developed under restricted topology assumptions, such as series-parallel (SP), non-series-parallel (NSP), planar, or non-planar structures. Logic-representation-based approaches [3, 7, 16] are constrained by the chosen Boolean representation and its mapping rules, often leading to SOP-biased SP structures or decision-diagram-driven pass-transistor logic (PTL) implementations [3]. Graph-based approaches [2, 12, 13, 17, 19] enlarge the topology space to include NSP structures and can further reduce transistor count, but they are typically heuristic, largely single-stage, and do not guarantee global optimality. SAT-based exact synthesis [10, 14, 21, 22] addresses the optimality gap and has gradually expanded from single-stage [22] to two-stage [14] and further to multi-stage and multi-output formulations [10, 21]. However, prior exact formulations still operate within predefined topology spaces. Stack height, multi-stage, and multi-output are important synthesis options. Increasing stage depth can expose lower-transistor-count decompositions [21]. Stack height strongly affects circuit performance [17], but prior works typically treat it either as a post-check [17, 22] or only in limited single-stage synthesis settings [14]. Multi-output synthesis can substantially reduce transistor count through logic sharing across outputs [2, 10, 12, 13, 21]. At the same time, exact SAT-based synthesis remains computationally expensive [10, 22]. Prior works have reduced this cost either by simplifying the SAT formulation [22] itself or by using guided search with external heuristics [10], but the runtime complexity remains substantially higher than that of heuristic methods, motivating the need for effective acceleration.

Motivated by these observations, we introduce a *free-topology* synthesis formulation in which internal nets generated in one stage may be used only by subnetworks in subsequent stages under an

acyclic (no-feedback) constraint. This allows inter-stage net sharing beyond predefined topology templates. Based on this formulation, we develop a SAT-based exact synthesis framework that jointly supports stack-height, multi-stage, and multi-output options as explicit constraints. To make this expanded exact-synthesis space practical, we further introduce several design-space pruning techniques. Our main contributions are as follows:

- We propose a *free-topology* synthesis formulation in which internal nets generated in one stage may be used only by subsequent stages under an acyclic constraint, enabling inter-stage sharing beyond predefined topology templates.
- We present the first SAT-based exact synthesis framework that jointly incorporates stack-height constraints into multi-stage transistor-network synthesis and supports a multi-output option.
- We introduce design-space pruning techniques for SAT-based exact synthesis to reduce computational complexity. These include basic structural constraints, symmetry-breaking constraints, and search-range pruning based on tighter internal-net bounds.
- We validate the proposed framework through extensive experiments, including transistor-count comparisons on 3-input P-class functions, the NSP benchmark, and multi-output functions, as well as 4-stack versus 2-stack studies quantifying transistor-count overhead and SPICE-based rise/fall balance improvements. We further present circuit-level delay/power/area (DPA) comparisons against prior 3-input implementations.

2 Preliminaries

In this section, we summarize the preliminaries used throughout the paper. Section 2.1 introduces the free-topology transistor-network model. Section 2.2 defines stage and stack height with a motivating example. Section 2.3 presents the static CMOS functional requirements underlying our synthesis formulation.

2.1 Free-Topology

We first illustrate the proposed *free-topology* concept, where internal nets generated in one stage may be reused only by subnetworks in subsequent stages, subject to an acyclic constraint. Figure 1

compares XOR3 synthesis results under different topology settings. Figure 1 (a) shows the Boolean equation and truth table. SP and NSP yield 26- and 22-transistor implementations in Fig. 1 (b) and (c), respectively, whereas *free-topology* yields a 16-transistor 3-stage implementation in Fig. 1 (d). Here, stage-1 nets nA and nB are reused by later stages, illustrating the inter-stage connectivity enabled by *free-topology*. The circuit also contains two inter-stage transmission-gate (TG) structures, another structural feature enabled by *free-topology*.

2.2 Stage and Stack Definitions with a Motivating Example

Logic stage is determined by the number of successive *gate-control* levels along a primary-input-to-output path. A circuit is 1-stage if the output is realized through a single connected diffusion network using only source/drain conduction. Whenever the output of one conduction subnetwork drives the gate(s) of transistors in another subnetwork, an additional stage is introduced. We model a multi-stage circuit as an *acyclic* dependency graph of internal nets, where the gate-control signals of stage d may depend only on primary inputs (PIs) and nets produced in earlier stages ($< d$) [3]. The *stage depth* of a synthesized solution is defined as the worst-case PI-to-output dependency depth. We define the **stack height** of a net under a given input vector as the minimum number of series-connected ON transistors on a valid conducting path from the corresponding supply rail (V_{DD} or V_{SS}) to that net. The stack height of a synthesized transistor network is the maximum of this value over all truth-table rows and required output and gate-driving nets. Stack height reflects the number of series devices on a required conducting path and therefore affects delay and rise/fall balance [6, 15, 17]. We call a synthesized transistor network $\mathcal{H}$ -stack if its stack height does not exceed $\mathcal{H}$. Figure 2 illustrates the interaction between stage depth and stack height using NOR4. Fig. 2 (a) shows a conventional 1-stage 4-stack implementation, whereas Fig. 2 (b) shows our 2-stack result, which realizes NOR4 in 3 stages with 14 transistors. Figs. 2 (c) and (d) compare the fall/rise output transition times and propagation delays, respectively, using ASAP7-based [5] SPICE simulation and the testbench in Fig. 10. The 4-stack design exhibits a large rise/fall skew, with mean transition and delay ratios of 0.22, whereas the 2-stack design improves them to 0.74 and 0.66. This example shows that tighter stack-height constraints can trade transistor count and stage depth for better rise/fall balance.

2.3 Static CMOS Functional Requirements

To guarantee that the synthesized transistor network implements a valid static digital CMOS function, we impose four functional requirements (FRs), illustrated in Fig. 3. Throughout this paper, logic '1' and '0' denote V_{DD} and V_{SS} , respectively.

FR1 (PI-gate-only). Primary inputs may drive only transistor gate terminals to avoid input-vector-dependent input capacitance. As illustrated in Fig. 3 (a), if a PI connects to a source/drain terminal, the input capacitance seen at that pin can vary with other input values. For example, when $B = 1$, TG turns on and both C_1 and C_2 contribute to the load seen by A ; when $B = 0$, TG turns off and only C_1 remains. This dynamic input capacitance, which depends on the input vector, makes standard timing sign-off unavailable.

FR2 (No-floating-gate). Any internal net used as a gate-driving signal must be connected to a valid logic level. In Fig. 3 (b) (left), when $A = 1$ and $B = 0$, the internal gate-driving net is connected to neither V_{DD} nor V_{SS} and therefore floats. This leaves the ON/OFF states of downstream transistors undefined, potentially causing

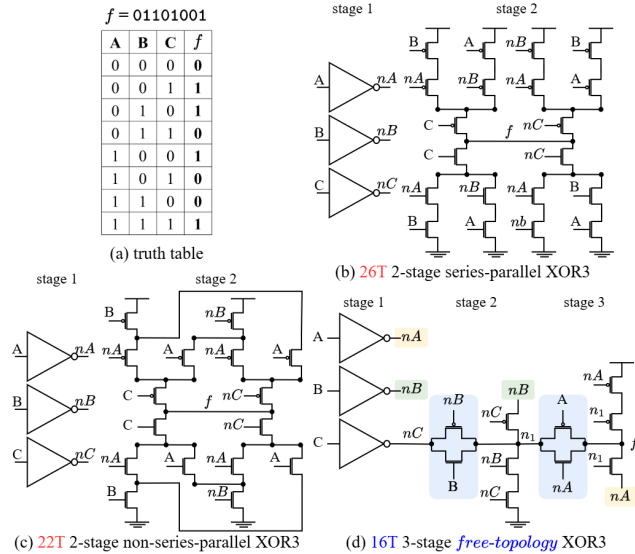


Figure 1: (a) Boolean equation and truth table of XOR3. (b) 2-stage SP XOR3 with 26 transistors. (c) 2-stage NSP XOR3 with 22 transistors. (d) Our 16-transistor 3-stage *free-topology* XOR3.

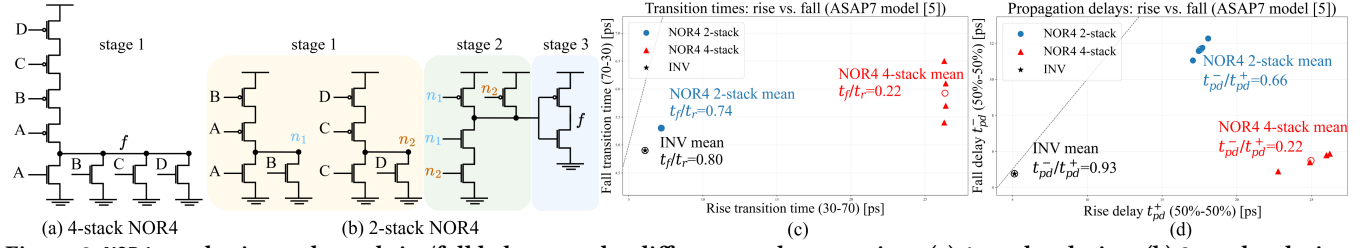


Figure 2: NOR4 synthesis results and rise/fall balance under different stack constraints. (a) 4-stack solution. (b) 2-stack solution with 3 stages. (c) Output-transition ratio comparison. The 2-stack design is closer to inverter-like balance. (d) Propagation-delay ratio comparison. The 2-stack design also shows more balanced rise/fall delay.

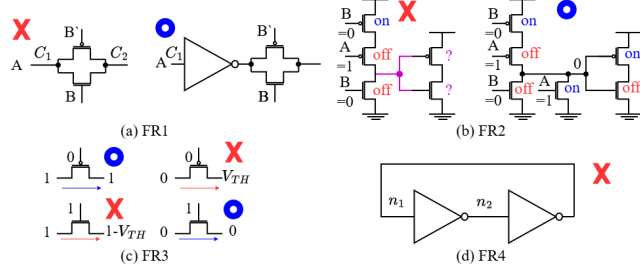


Figure 3: Functional requirements for valid static digital CMOS behavior in synthesized transistor networks: (a) FR1: PIs are restricted to gate terminals to avoid input-vector-dependent input capacitance; (b) FR2: gate-control nets must not float; (c) FR3: weak logic delivery through threshold-loss paths is disallowed; (d) FR4: feedback loops are prohibited.

shorts, floating outputs, or incorrect logic values. Therefore, as shown in Fig. 3 (b) (right), additional transistors are needed to prevent floating gate-control nets.

FR3 (No-weak-level-delivery). Gate-driving nets must carry full-swing logic levels. An NMOS cannot deliver a strong logic ‘1’, and a PMOS cannot deliver a strong logic ‘0’ because of threshold-voltage loss. As shown in Fig. 3 (c), an NMOS-passed ‘1’ is limited near $V_{DD} - V_{TN}$, while a PMOS-passed ‘0’ is limited near $|V_{TP}|$. Such degraded levels can cause functional failure, so weak ‘1’/‘0’ delivery is disallowed on gate-driving nets.

FR4 (Acyclic network). The synthesized transistor network must be acyclic. As shown in Fig. 3 (d), feedback-loop nets such as n_1 and n_2 may admit PI-independent internal assignments that are not uniquely determined by the input vector. A SAT solver can exploit such internally uncontrolled values to satisfy constraints without representing valid input-driven circuit behavior. We therefore constrain the synthesized network to be acyclic.

3 Proposed Method

This section presents our *free-topology* transistor-network synthesis framework under stack-height constraints with stage-depth and multi-output options. The formulation consists of four components: transistor-network modeling (Section 3.1), multi-stage-aware stack-height verification (Section 3.2), acyclicity constraints (Section 3.3), and design-space pruning techniques (Section 3.4). Table 1 summarizes the notation used throughout this section, and Fig. 4 shows the overall synthesis flow. Given a logic specification f , a maximum stage-depth bound $|D|^{max}$, and a stack-height bound $\mathcal{H}$, the solver searches for a valid transistor network by progressively increasing the transistor budget, stage depth, and internal-net budget until a feasible solution is found. Here, f may represent either a single-output or a multi-output function.

Table 1: Notation used in the proposed framework.

Notation	Description
$ P $	Number of primary inputs
$ T $	Number of transistors
$ P_p $	Number of PMOS transistors
$ N_p $	Number of NMOS transistors
$ D $	Number of stages
$\mathcal{H}$	Maximum allowed stack height
$ N $	Number of nets ($ N_{int} + N_{out} + N_{power} $)
n_j	The j -th internal net
t_i	The i -th transistor
$source_{t_i, n_j}$	1 if the source terminal of t_i is connected to n_j
$gate_{t_i, n_j}$	1 if the gate terminal of t_i is connected to n_j
$drain_{t_i, n_j}$	1 if the drain terminal of t_i is connected to n_j
$ON_p^{(k)}(t_i)$	1 if PMOS transistor t_i is ON in the k -th truth-table row
$ON_n^{(k)}(t_i)$	1 if NMOS transistor t_i is ON in the k -th truth-table row
f	Logic specification (single-output or multi-output)
f_j	The j -th output function / output net
$o_j^{(k)}$	Output bit of f_j in the k -th truth-table row

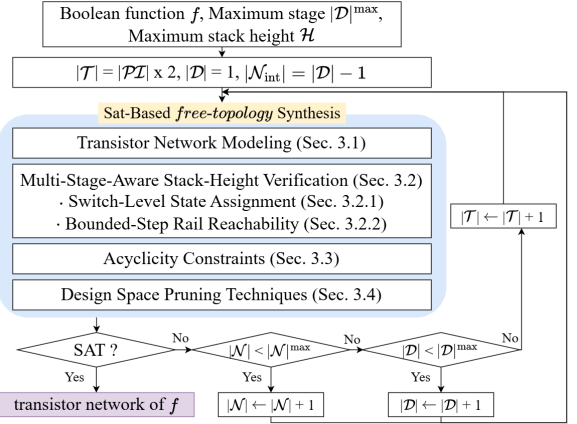


Figure 4: Overview of our proposed framework.

3.1 Transistor Network Modeling

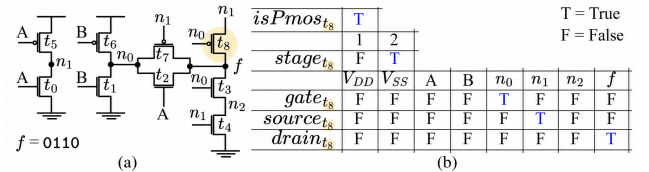


Figure 5: (a) Synthesized transistor network of 9T XOR2. (b) SAT variable assignments for t_8 .

Figure 5 illustrates the transistor modeling used in our framework: Fig. 5 (a) shows a synthesized XOR2 network, and Fig. 5 (b) lists the Boolean assignments for a representative transistor t_8 .

As shown in Fig. 4, for each SAT instance, we instantiate a fixed number of transistor templates. Each transistor is characterized by

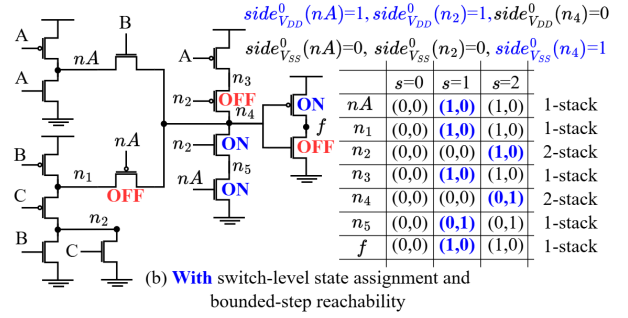
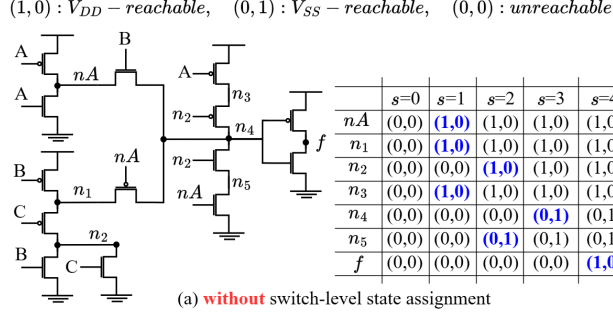


Figure 6: Motivation for multi-stage-aware stack-height verification using our 2-stack circuit for $f = 10000011$ under truth-table row $k = 0$, i.e., $(A, B, C) = (0, 0, 0)$. (a) Without switch-level state assignment, the stack height of the output cannot be determined directly. (b) With switch-level state assignment, later-stage gate control is fixed, and bounded-step rail reachability can be used to verify that the required nets satisfy the stack-height bound. The reachability table lists $(reach_{V_{DD}}^{(k,s)}(n_i), reach_{V_{SS}}^{(k,s)}(n_i))$.

five groups of decision variables: *isPmos*, *stage*, *gate*, *source*, and *drain*. Here, *isPmos* determines transistor polarity, *stage* assigns the transistor to a stage in the multi-stage network, and *source*, *gate*, and *drain* specify the net connected to each terminal. This modeling is intentionally general: rather than fixing a predefined topology template, the solver determines transistor polarity, stage assignment, and terminal connectivity directly, thereby enabling *free-topology* transistor-network synthesis. Since our formulation simultaneously synthesizes the pull-up and pull-down networks, *isPmos* is particularly important when the total transistor budget is odd. The *stage* variables are later used to enforce the acyclicity (no-feedback) constraint in Section 3.3. Given the logic specification f , the solver evaluates the transistor network under the primary-input (PI) assignment of each truth-table row k and searches for a solution whose outputs match $o_j^{(k)}$ for every row k and every output f_j . We first impose basic net-usage constraints to enforce the intended static-CMOS semantics. Primary inputs are allowed only on gate terminals (FR1), V_{DD} and V_{SS} only on source terminals, and outputs only on drain terminals, while internal nets n_i may appear on gate, source, or drain terminals. We further require each internal net n_i to be generated by at least one drain connection and to be used either as a source or as a gate-driving net, so as to avoid dangling or floating internal nets:

$$\left(\sum_{t_j \in \mathcal{T}} drain_{t_j, n_i} \geq 1 \right) \wedge \left(\sum_{t_j \in \mathcal{T}} (source_{t_j, n_i} + gate_{t_j, n_i}) \geq 1 \right).$$

This constraint ensures that every internal net participates in a meaningful conduction or control relationship within the synthesized network.

3.2 Multi-Stage-Aware Stack-Height Verification

We verify the $\mathcal{H}$ -stack condition in multi-stage networks by first assigning switch-level states to internal nets and then applying bounded-step rail reachability. Since later-stage transistor ON/OFF states depend on earlier-stage internal nets, active paths from the power rails to internal or output nets cannot be determined until those gate-control states are fixed. Figure 6 illustrates this issue using our 2-stack circuit for $f = 10000011$ under truth-table row $k = 0$. In Fig. 6 (a), gate-control signals such as nA and n_2 are resolved only through earlier-stage evaluation, making the stack height of later nets such as n_4 and f difficult to determine directly. In Fig. 6 (b), switch-level state assignment first fixes gate ON/OFF

conditions, and bounded-step rail reachability then verifies that all required nets are generated within two ON-transistor traversals ($s=2$), confirming the 2-stack bound. The following two parts describe these constraints.

3.2.1 Switch-Level State Assignment. This step determines the switch-level state of each net for every truth-table row by propagating V_{DD} -side and V_{SS} -side connectivity. These states are then used to determine later-stage gate control before stack-height verification. For each net n_i and truth-table row k , we introduce two Boolean variables, $side_{V_{DD}}^{(k)}(n_i)$ and $side_{V_{SS}}^{(k)}(n_i)$, indicating whether n_i is connected to the V_{DD} side or the V_{SS} side, respectively. The pair $(side_{V_{DD}}^{(k)}(n_i), side_{V_{SS}}^{(k)}(n_i))$ can be (1, 0), (0, 1), or (0, 0), while (1, 1) is forbidden because it implies simultaneous connection to both rails. The supply rails are anchored as

$$side_{V_{DD}}^{(k)}(V_{DD}) = 1, \quad side_{V_{SS}}^{(k)}(V_{SS}) = 1,$$

and the PIs are fixed according to the k -th row of the truth table. These variables provide provisional states for internal nets. If an internal net is used as a gate-driving signal, the floating state (0, 0) is disallowed, enforcing FR2. Transistor ON/OFF follows standard CMOS behavior: a PMOS turns ON when its gate is on the V_{SS} side, and an NMOS turns ON when its gate is on the V_{DD} side. FR3 is enforced at the propagation level: an ON PMOS propagates only V_{DD} -side connectivity, and an ON NMOS propagates only V_{SS} -side connectivity across source/drain terminals. For readability, let $src(t_i)$ and $drn(t_i)$ denote the nets selected by $source_{t_i, n}$ and $drain_{t_i, n}$, respectively. Then, for each transistor t_i , we enforce

$$ON_p^{(k)}(t_i) \Rightarrow side_{V_{DD}}^{(k)}(src(t_i)) = side_{V_{DD}}^{(k)}(drn(t_i)),$$

$$ON_n^{(k)}(t_i) \Rightarrow side_{V_{SS}}^{(k)}(src(t_i)) = side_{V_{SS}}^{(k)}(drn(t_i)).$$

The opposite propagation rules are not imposed, so weak delivery is excluded from valid logic propagation. A separate conservative short check rules out possible rail-to-rail short paths through ON transistors.

3.2.2 Bounded-Step Rail Reachability. This step checks whether the switch-level states from Section 3.2.1 can be physically realized within the stack-height bound $\mathcal{H}$. For each truth-table row k , step s , and net n_i , we define $reach_{V_{DD}}^{(k,s)}(n_i)$ and $reach_{V_{SS}}^{(k,s)}(n_i)$, which indicate whether n_i is reachable from V_{DD} or V_{SS} , respectively,

within at most s ON-transistor traversals. At $s = 0$, only the supply rails are reachable:

$$reach_{V_{DD}}^{(k,0)}(V_{DD}) = 1, \quad reach_{V_{SS}}^{(k,0)}(V_{SS}) = 1,$$

while all other nets are initialized to 0. Reachability is propagated step by step. Each net preserves its previous reachability and additionally inherits reachability from adjacent nets connected through ON transistors, where only ON PMOS transistors are used for V_{DD} -reachability and only ON NMOS transistors are used for V_{SS} -reachability. After $\mathcal{H}$ steps, outputs must be reachable from the correct rail:

$$o_j^{(k)} = 1 \Rightarrow reach_{V_{DD}}^{(k,\mathcal{H})}(f_j), \quad o_j^{(k)} = 0 \Rightarrow reach_{V_{SS}}^{(k,\mathcal{H})}(f_j).$$

In addition, any internal net used as a gate-driving signal must be reachable from at least one valid rail within $\mathcal{H}$ steps:

$$used_g(n_i) \equiv \left(\sum_{t \in \mathcal{T}} gate_{t,n_i} \geq 1 \right),$$

$$used_g(n_i) \Rightarrow reach_{V_{DD}}^{(k,\mathcal{H})}(n_i) \vee reach_{V_{SS}}^{(k,\mathcal{H})}(n_i).$$

Finally, the switch-level state and bounded-step reachability must agree for every gate-driving internal net:

$$used_g(n_i) \Rightarrow side_{V_{DD}}^{(k)}(n_i) = reach_{V_{DD}}^{(k,\mathcal{H})}(n_i),$$

$$used_g(n_i) \Rightarrow side_{V_{SS}}^{(k)}(n_i) = reach_{V_{SS}}^{(k,\mathcal{H})}(n_i).$$

These constraints remove spurious solutions in which later-stage gate control relies on internal-net states that cannot be generated within the prescribed stack-height bound. Thus, both outputs and gate-driving internal nets are guaranteed to be realizable within $\mathcal{H}$ ON-transistor traversals for every truth-table row.

3.3 Acyclicity Constraints

To enforce acyclicity (FR4), dependencies follow stage order: gate-driving nets must come from earlier stages, and later-stage internal nets cannot reconnect to earlier-stage source/drain terminals.

- **Gate dependency from earlier stages.** If an internal net n drives the gate of transistor t_i , and n is generated as the drain of transistor t_j , then t_j must belong to a strictly earlier stage:

$$(gate_{t_i,n} = 1 \wedge drain_{t_j,n} = 1) \Rightarrow stage(t_j) < stage(t_i).$$

- **No backward endpoint reconnection.** If an internal net is generated as the drain net of a later-stage transistor, it cannot appear as the source or drain net of an earlier-stage transistor. Formally, for $stage(t_e) < stage(t_l)$,

$$drain_{t_e,n_i} = 1 \Rightarrow source_{t_e,n_i} = 0 \wedge drain_{t_e,n_i} = 0, \quad \forall n_i \in \mathcal{N}_{int}.$$

3.4 Design-Space Pruning Techniques

To reduce redundant and structurally implausible exploration, we apply pruning at three levels: basic structural constraints on transistor/net usage, symmetry-breaking constraints on equivalent transistor assignments, and search-range pruning on the number of internal nets.

3.4.1 Basic structural constraints. We first impose structural constraints to eliminate invalid or redundant transistor configurations.

- **Output-at-last-stage.** If a transistor drives an output net, it must belong to the last stage:

$$drain_{t_i,f_j} = 1 \Rightarrow stage(t_i) = |\mathcal{D}|.$$

- **Stage-1 anchoring.** The first stage must contain at least one PMOS transistor connected to V_{DD} and at least one NMOS transistor connected to V_{SS} :

$$\sum_{t_i \in \mathcal{T}} source_{t_i,V_{DD}} \cdot isPmos_{t_i} \geq 1, \quad \sum_{t_i \in \mathcal{T}} source_{t_i,V_{SS}} \cdot (1 - isPmos_{t_i}) \geq 1.$$

- **Stage-1 PI-gate-only.** Only primary inputs may drive gate terminals in the first stage:

$$stage(t_i) = 1 \Rightarrow \sum_{n \in \mathcal{P}_I} gate_{t_i,n} = 1.$$

- **Polarity presence per stage.** Each stage must contain at least one PMOS transistor and one NMOS transistor:

$$\forall k \in \{1, \dots, |\mathcal{D}|\}, \quad \sum_{t_i: stage(t_i)=k} isPmos_{t_i} \geq 1, \quad \sum_{t_i: stage(t_i)=k} (1 - isPmos_{t_i}) \geq 1.$$

3.4.2 Symmetry-breaking constraints. We impose symmetry-breaking constraints on transistor and internal-net assignments.

- **Lexicographic transistor ordering.** To remove permutation symmetry, consecutive same-polarity transistors are ordered by stage, gate, source, and drain:

$$stage(t_i) \leq stage(t_{i+1}),$$

$$stage(t_i) = stage(t_{i+1}) \Rightarrow gate(t_i) \leq gate(t_{i+1}),$$

$$gate(t_i) = gate(t_{i+1}) \Rightarrow source(t_i) \leq source(t_{i+1}),$$

$$source(t_i) = source(t_{i+1}) \Rightarrow drain(t_i) \leq drain(t_{i+1}).$$

- **Polarity-aware stage ordering.** Within each polarity group, stage assignments are constrained to be nondecreasing:

$$isPmos_{t_i} = isPmos_{t_{i+1}} \Rightarrow stage(t_i) \leq stage(t_{i+1}).$$

- **Source-drain orientation.** To remove symmetric endpoint assignments, we impose a fixed source-drain orientation:

$$source(t_i) \leq drain(t_i).$$

3.4.3 Search-Range Pruning. The allowable internal-net count creates a large synthesis design space under a fixed transistor budget. Prior works [21] use a loose range based mainly on transistor count, which includes many structurally implausible configurations. We therefore tighten the search range for the number of internal nets. For the lower bound, we use

$$|\mathcal{N}_{int}|^{\min} = |\mathcal{D}| - 1.$$

This follows directly from our stage definition: a multi-stage circuit with $|\mathcal{D}| > 1$ must contain at least one inter-stage connection, and therefore at least $|\mathcal{D}| - 1$ internal nets are required.

For the upper bound, we consider two reference cases. First, under the planar primal-dual interpretation used in L2L [14], the PMOS and NMOS networks form a primal-dual pair with $|\mathcal{T}_p| = |\mathcal{T}_n|$. Let the primal PMOS graph have $|\mathcal{N}|$ nodes and $|\mathcal{T}_p|$ edges. After adding one auxiliary edge between V_{DD} and f , Euler's formula [8] gives

$$F = (|\mathcal{T}_p| + 1) - |\mathcal{N}| + 2 = |\mathcal{T}_p| - |\mathcal{N}| + 3.$$

Since each primal face corresponds to one dual node, the total number of nodes in the primal-dual pair is

$$|\mathcal{N}| + F = |\mathcal{T}_p| + 3.$$

Thus, the total net count in the planar primal-dual case is bounded by $|\mathcal{T}_p| + 3$.

Second, we consider the non-planar case, where the PMOS and NMOS networks are treated as independent connected dangling-free non-planar graphs. By Kuratowski's theorem [8], non-planarity

is characterized by a subdivision of K_5 or $K_{3,3}$. Among these, $K_{3,3}$ gives the largest number of nodes for a fixed edge budget under the dangling-free condition. Starting from $K_{3,3}$, the node count can be further increased only by subdividing edges, i.e., by inserting series transistors, which adds one node per added edge. Therefore, for a non-planar graph with m edges, the maximum number of nodes is $m - 3$. In our setting, adding one auxiliary edge between a supply rail and f gives $m = |\mathcal{T}_p| + 1$ on the PMOS side and $m = |\mathcal{T}_n| + 1$ on the NMOS side, so the total net count is bounded by

$$(|\mathcal{T}_p| + 1 - 3) + (|\mathcal{T}_n| + 1 - 3) = |\mathcal{T}| - 4.$$

Combining these two cases, we bound the total net count by

$$|\mathcal{N}|^{\max} = \max(|\mathcal{T}_p| + 3, |\mathcal{T}| - 4).$$

Finally, since four nets are fixed externally— V_{DD} , f on PMOS, and V_{SS} , f on NMOS—we convert this to the internal-net bound

$$|\mathcal{N}_{\text{int}}|^{\max} = |\mathcal{N}|^{\max} - 4 = \max(|\mathcal{T}_p| - 1, |\mathcal{T}| - 8).$$

Although the upper bound is derived from single-stage graph structures, it remains valid for the multi-stage setting. Additional stages may increase the minimum number of required inter-stage nets, but they do not increase the graph-structural maximum net count under a fixed transistor budget. This tighter range,

$$|\mathcal{D}| - 1 \leq |\mathcal{N}_{\text{int}}| \leq \max(|\mathcal{T}_p| - 1, |\mathcal{T}| - 8),$$

substantially reduces the internal-net search space while retaining the structurally relevant solutions.

4 Experimental Results

We implemented the framework in C++. Each SAT instance is emitted as a CNF file and solved using Kissat [11]. To mitigate runtime explosion in UNSAT cases, each run is given a timeout of $|\mathcal{P}|$ hours. Unless otherwise stated, the stack-height bound is set to $\mathcal{H} = 4$, following [17]. We constrain the PMOS/NMOS count imbalance as $||\mathcal{T}_p| - |\mathcal{T}_n|| \leq 2$, which supports odd transistor budgets and allows up to a two-device polarity imbalance; Fig. 7 illustrates its benefit. Circuit-level simulations use Synopsys HSPICE *T-2022.06-4* with the ASAP7 PDK [5]. All experiments run on a Linux workstation with a 32-core Intel Xeon E5-2620 CPU and 256 GB RAM. The experimental artifacts, including solver logs, SPICE cells, and HSPICE input decks, are publicly available in our GitHub repository [1].

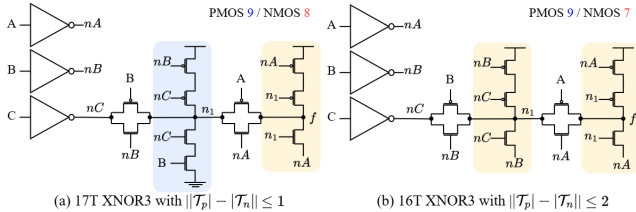


Figure 7: Effect of the PMOS/NMOS-count imbalance bound on XNOR3: (a) best solution under $||\mathcal{T}_p| - |\mathcal{T}_n|| \leq 1$; (b) optimal 16-transistor solution under $||\mathcal{T}_p| - |\mathcal{T}_n|| \leq 2$.

4.1 Transistor Count Evaluation

We evaluate transistor count on three benchmarks: 3-input P-class functions, the 53 handcrafted NSP networks [9], and multi-output functions [12].

Table 2: Aggregate comparison of SAT-based transistor-network synthesis approaches on 78 3-input P-class functions (excluding the always-0 and always-1 functions from the original 80-function set).

Approach	Scheme [†]	$\mathcal{H}$	$ \mathcal{D} $	Topology [‡]	Coverage	Total $ \mathcal{T} $ (Ratio)
MiniTNtk [22]	<i>P/N seq.</i>	∞	$\leq 2^\circ$	Arbitrary	100%	1036 (1.000)
MuSTNet [21]	<i>P/N sim.</i>	∞	∞	Arbitrary	100%	838 (0.809)
L2L [14]	<i>P-only</i>	≤ 2	≤ 2	Arbitrary	74%	N/A [§]
L2L [14]	<i>P-only</i>	≤ 4	≤ 2	SP-only	100%	896 (0.865)
L2L [14]	<i>P-only</i>	≤ 4	≤ 2	Arbitrary	100%	884 (0.853)
Ours	<i>P/N sim.</i>	≤ 2	≤ 4	<i>free-topology</i>	100%	908 (0.876)
Ours	<i>P/N sim.</i>	≤ 4	≤ 4	<i>free-topology</i>	100%	820 (0.792)

[†] **Synthesis scheme.** *P-only* solves only one network (e.g., pull-up) in SAT and then derives the complementary network from the solved result. *P/N seq.* synthesizes P and N networks sequentially, while *P/N sim.* synthesizes P and N networks simultaneously.

[‡] **Topology.** *Arbitrary* allows both series-parallel (SP) and non-series-parallel (NSP) transistor network structures.

[°] MiniTNtk is considered partially 2-stage because inverted primary inputs are allowed as gate-control signals; although the transistor network itself is single-stage, generating an inverted input requires an additional inverter stage.

[§] N/A because this setting covers only 74% of the benchmark set (i.e., no meaningful aggregate total #tr over all 78 functions).

4.1.1 3-input P-class Functions. We compare recent SAT-based transistor-network synthesis approaches on the 78 3-input P-class benchmark functions (excluding the always-0 and always-1 cases from the original 80-function set). Table 2 shows that under a 4-stack constraint, our *free-topology* setting achieves a total of 820 transistors, reducing the count over MiniTNtk [22] by 21% and MuSTNet [21] by 2%. Under the stricter 2-stack constraint, our *free-topology* synthesis achieves 100% coverage, whereas the L2L [14] baseline with 2-stack and 2-stage constraints covers only 74% of the functions. This full coverage is achieved by increasing stage depth, resulting in a total transistor count of 908.

Table 3: Total number of transistors in the NMOS pull-down networks for the 53 NSP functions [9].

Metric	[9] (opt.)	[16]	[6]	[18]	[13]	[17]	[22]	Ours	Bound [17]	Bound [22]	Bound Ours
$ \mathcal{T}_n $	356	487	503	516	543	359	356	356	383	379	371

4.1.2 53 Handcrafted NSP Transistor Networks. We evaluate whether our framework can recover known optimal NSP networks and synthesize bounded-stack alternatives on the 53 handcrafted NSP benchmark functions [9]. We use two settings: $\mathcal{H} = 6$ to match the known 1-stage optimum, and $\mathcal{H} = 4$ for comparison with bounded-stack prior work [17, 22]. Table 3 shows that our method matches the known optimum of 356 transistors under $\mathcal{H} = 6$, and increases to 371 under $\mathcal{H} = 4$. Following the benchmark convention, this count includes only NMOS pull-down networks; enforcing the same 4-stack constraint on PMOS networks gives 379. Even under this tighter setting, our result uses no more transistors than prior bounded-stack approaches [17, 22].

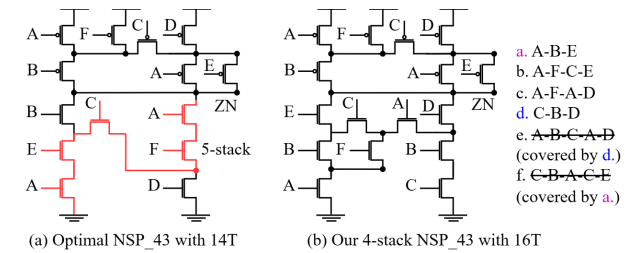


Figure 8: NSP_43 [9] under different stack-height bounds: (a) 14-transistor optimum with maximum stack height 5; (b) 16-transistor circuit under $\mathcal{H} = 4$.

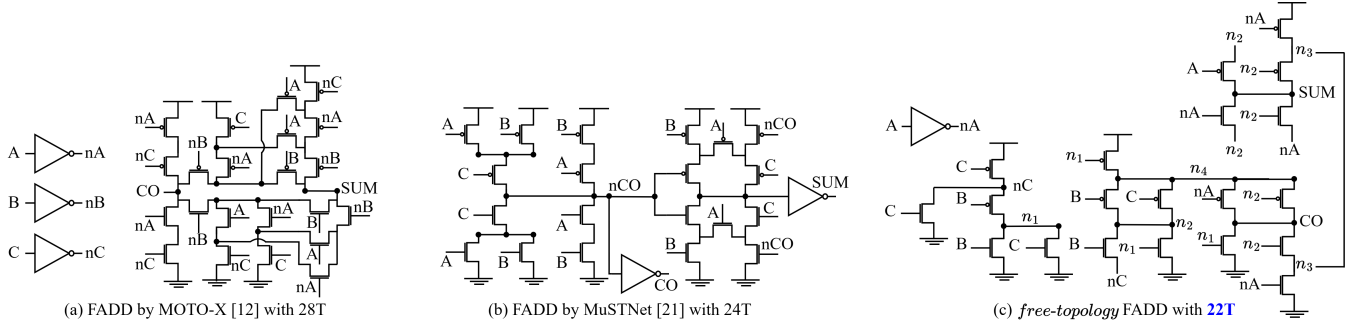


Figure 9: Three full adder (FADD) circuit implementations: (a) 28T from [12]; (b) 24T from [21]; (c) Our 22T *free-topology* circuit.

Figure 8 illustrates how the tighter stack-height bound changes the NSP_43 [9] network. Our $\mathcal{H} = 6$ synthesis recovers the 14-transistor circuit in Fig. 8 (a), whereas $\mathcal{H} = 4$ yields the 16-transistor circuit in Fig. 8 (b) by adding two NMOS transistors. Although Fig. 8 (b) contains two five-series paths, shorter conducting paths provide the same required connections; therefore, these paths do not increase the stack-height requirement.

4.1.3 Multi-Output Functions. We evaluate selected multi-output complex gates from the MOTO-X benchmark [12] and compare against graph-based [12] and SAT-based methods [10, 21]. **Table 4: Transistor count evaluation on representative multi-output benchmark cases [12].**

Functions	#in/#out	MOTO-X [12]	Hybrid [10]	MuSTNet [21]	Ours ($\mathcal{H} \leq 4$)	
					$ \mathcal{T} $	$ \mathcal{D} $
STE1	3/2	24	24	22	21	3
FADD	3/2	28	27	24	22	3
BAR1	3/3	25	27	24	24	2
STE3*	3/3	28	23	23	25	3
AGR2	4/2	28	28	26	26	3
GUR2	4/2	22	21	20	18	2
BAR2	4/3	35	36	30	30	3
AGR1	4/3	34	30	28	27	2
SPEC	4/4	44	55	44	43	4

* Our STE3 result uses the original MOTO-X specification [12]:

$F_1 = \Sigma m(0, 2, 3, 4, 5, 7)$, $F_2 = \Sigma m(1, 5, 6, 7)$, and $F_3 = \Sigma m(1, 2, 5, 7)$. In [10], $F_1 = \Sigma m(1, 3, 4, 5, 7)$ was used instead, while F_2 and F_3 remain unchanged; the exact STE3 specification used in [21] could not be determined.

ble 4 reports representative cases under our bounded-stack setting $\mathcal{H} \leq 4$. Our method improves the transistor count in several cases and matches the best-reported count in others, while explicitly reporting the stage depth $|\mathcal{D}|$ required by the synthesized multi-stage solutions. These results demonstrate that the proposed framework can synthesize compact multi-output transistor networks under multi-stage and 4-stack conditions. Figure 9 shows a representative example, the full adder (FADD). Compared with the 28-transistor circuit in [12] and the 24-transistor circuit in [21], our method synthesizes a 22-transistor *free-topology* circuit with substantial inter-stage net sharing (nC , n_2 and n_4).

4.2 Circuit-Level Comparison on 3-Input P-Class Functions

To evaluate whether transistor-count reduction translates into circuit-level benefit, we compare our 4-stack cells against prior 3-input implementations from MiniTNtk [22] and L2L [14]. Among the 78 non-constant 3-input P-class functions, 10 are functionally one- or two-input cases. We exclude these and evaluate the remaining 68 functions that depend on all three inputs. Since this comparison aims to quantify the circuit-level impact of transistor-count

reduction, we focus on reduced-count cases, yielding 52 comparable cells against MiniTNtk and 22 against L2L. Figure 10 shows the

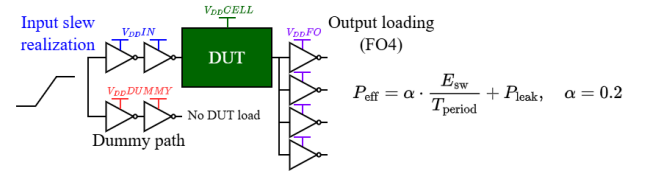


Figure 10: SPICE testbench for delay and power measurement, with two input inverters for slew shaping, four output inverters for FO4-like loading, and separate power sources to isolate DUT (Design Under Test) power.

SPICE testbench. The DUT is driven by two cascaded input inverters and loaded by four output inverters. Separate power sources isolate DUT power, and dummy-inverter subtraction removes inverter power unrelated to DUT loading. We define

$$E_{sw} = E_{cell} + (E_{inv,cell} - E_{inv,dummy}) - P_{leak} T_{window},$$

and

$$P_{eff} = \alpha \cdot \frac{E_{sw}}{T_{per}} + P_{leak}, \quad \alpha = 0.2.$$

Figure 11 compares delay, effective power, and transistor-count-

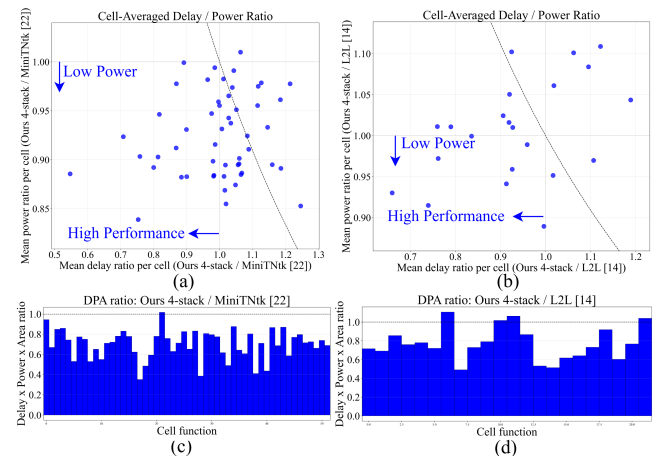


Figure 11: Circuit-level comparison of our 4-stack cells with MiniTNtk [22] and L2L [14]: (a,b) delay and effective-power ratios; (c,d) delay-power-area (DPA) ratios.

based area. Each cell-level ratio is computed by averaging arc-wise ratios that are one-to-one matched. In Figs. 11 (a) and (b), the curve

$y = 1/x$ marks equal delay-power product; points below the curve favor Ours 4-stack. Our method improves the delay-power product for 39 of 52 MiniTNTk cells (75.0%) and 15 of 22 L2L cells (68.2%). In Figs. 11 (c) and (d), our method improves DPA for all but 1 MiniTNTk function and all but 4 L2L functions, with best delay-power-area (DPA) improvements of 64.8% and 50.9%, respectively.

4.3 Comparison of 4-Stack and 2-Stack Synthesis on 3-Input P-Class Functions

This section compares 4-stack and 2-stack synthesis results for 3-input P-class functions. The comparison covers synthesis statistics, representative cases, and SPICE-based rise/fall balance.

4.3.1 Synthesis Statistics and Case Study. Under the stricter 2-stack constraint, our framework still covers all 78 evaluated 3-input P-class functions, but it requires additional stage depth and transistor count for some of them. Table 5 shows representative penalty

Table 5: Representative penalty patterns from 4-stack to 2-stack synthesis.

Function	4-stack		2-stack		Penalty	
	$ T $	$ D $	$ T $	$ D $	$\Delta T $	$\Delta D $
10000000	6	1	10	3	+4	+2
11111110	6	1	10	3	+4	+2
00011000	10	2	16	3	+6	+1
10111101	10	2	16	3	+6	+1
01101000	15	2	19	3	+4	+1
10011110	15	2	19	3	+4	+1
10000001	12	3	17	3	+5	0
01101110	12	2	12	3	0	+1

patterns when moving from 4-stack to 2-stack. The first two rows show the most severe stage increase, where 1-stage solutions become 3-stage solutions. The middle rows show cases where both transistor count and stage depth increase together. The seventh row shows a case where only the transistor count increases, while the last row shows a case where the function is still covered by increasing only the stage depth without extra transistors. Figure 12

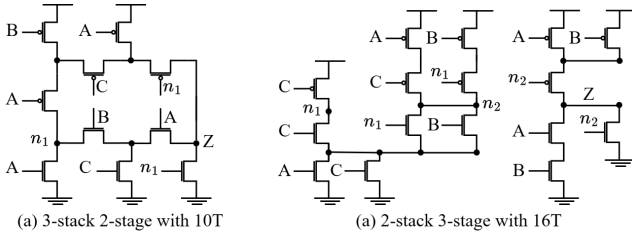


Figure 12: Synthesized circuits for truth table 00011000: (a) 10T 3-stack, 2-stage circuit found under the 4-stack bound; (b) 16T 2-stack, 3-stage circuit found under the 2-stack bound.

shows a representative case. The 4-stack setting realizes 00011000 with a 10T, 2-stage circuit, whereas the 2-stack setting requires a 16T, 3-stage circuit.

4.3.2 SPICE-Based Rise/Fall Balance Comparison. We use SPICE simulation to compare rise/fall balance between the 2-stack and 4-stack solutions, measuring both output transition time and propagation delay. Figure 13 compares 364 paired rise/fall measurements for each setting, where each pair matches rise and fall arcs of the same output pin under the same simulation condition. Overall, the 2-stack solutions show better rise/fall balance, reducing

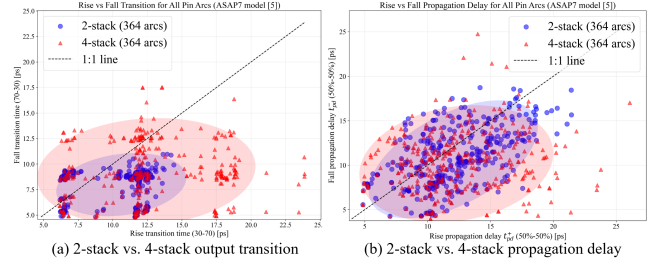


Figure 13: SPICE-based rise/fall balance comparison: (a) transition time; (b) propagation delay. The x/y axes show rise/fall values; blue circles/red triangles denote 2-stack/4-stack locations. Points closer to the diagonal indicate better balance.

the mean absolute rise/fall difference by 24.2% for transition time $4.29 \rightarrow 3.25$ ps and 22.8% for propagation delay $3.90 \rightarrow 3.01$ ps; the worst-case differences are also reduced by 61.2% ($18.18 \rightarrow 7.06$ ps) and 39.1% ($16.68 \rightarrow 10.16$ ps), respectively.

4.4 Effectiveness of Design-Space Pruning

This section evaluates the impact of design-space pruning on synthesis runtime. Following Fig. 4, we measure runtime on XOR3, XNOR3, and FADD at the transistor counts where SAT solutions are obtained, i.e., 16 for XOR3/XNOR3 and 22 for FADD. We use a 3-hour timeout per run; #TO denotes timed-out runs, and N. RT denotes total runtime normalized to the full-range baseline. Ta-

Table 6: Effectiveness of design-space pruning. “Full range” denotes the baseline without pruning, “SB” applies symmetry breaking in Sec. 3.4.2, and “SB+SRP” additionally applies search-range pruning in Sec. 3.4.3.

Cell ($ T $)	Full range			SB			SB+SRP		
	#Runs	#TO	N. RT	#Runs	#TO	N. RT	#Runs	#TO	N. RT
XOR3 (16)	28	15	1.000	26	5	0.401	19	0	0.023
XNOR3 (16)	26	12	1.000	26	5	0.492	19	0	0.033
FADD (22)	38	17	1.000	38	14	0.851	31	7	0.464

ble 6 shows that SB reduces normalized runtime by 59.9%, 50.8%, and 14.9% for XOR3, XNOR3, and FADD, respectively. Adding search-range pruning (SB+SRP) further removes redundant runs in the design-space sweep and reduces timed-out runs, improving the runtime reductions to 97.7%, 96.7%, and 53.6%.

5 Conclusion

This paper presented a SAT-based exact synthesis framework for *free-topology* multi-stage transistor networks under stack-height constraints. By allowing inter-stage net sharing under an acyclic constraint, the framework enlarges the transistor-network solution space beyond predefined topology templates. Experimental results show that the multi-stage and stack-height options enable diverse circuit solutions with different transistor counts and structural trade-offs, whose circuit-level characteristics are further evaluated via SPICE simulation. The results also demonstrate the effectiveness of design-space pruning in reducing runtime.

The main challenge is scalability. While the proposed formulation can generate compact circuits while enforcing the required static-CMOS validity constraints, the enlarged *free-topology* search space can lead to high SAT runtime. Future work will explore more scalable formulations and stronger pruning techniques for larger functions and library-level strategies for identifying and targeting practically useful cells.

References

- [1] 2026. Free-Topology-TNS-ICCAD2026. <https://github.com/b8kang/Free-Topology-TNS-ICCAD2026>
- [2] Anup Kumar Biswas and Dimitri Kagaris. 2025. Reducing transistor count in CMOS logic design through clustering and library-independent multiple-output logic synthesis. *IEEE Trans. CAD* 44, 8 (2025), 3046–3059. doi:10.1109/TCAD.2025.3538492
- [3] Buch, Narayan, and Newton. 1997. Logic synthesis for large pass transistor circuits. In *Proc. ICCAD*. 663–670. doi:10.1109/ICCAD.1997.643609
- [4] Chung-Kuan Cheng, Bill Lin, Byeonggon Kang, and Yucheng Wang. 2023. The Scope and Challenges of Scaling in Advanced Technologies. In *Proc. SLIP*. 1–8. doi:10.1145/3632409.3632841
- [5] Lawrence T Clark, Vinay Vashishtha, Lucian Shifren, Aditya Gujja, Saurabh Sinha, Brian Cline, Chandrasekaran Ramamurthy, and Greg Yeric. 2016. ASAP7: A 7-nm finFET predictive process design kit. *Microelectronics Journal* 53 (2016), 105–115. doi:10.1016/j.mejo.2016.04.006
- [6] Leomar Soares da Rosa, Andre Inacio Reis, Renato Perez Ribas, Felipe de Souza Marques, and Felipe Ribeiro Schneider. 2007. A comparative study of CMOS gates with minimum transistor stacks. In *Proceedings of the 20th annual conference on Integrated circuits and systems design*. 93–98. doi:10.1145/1284480.1284511
- [7] Leomar S da Rosa Junior, Felipe S Marques, Tiago MG Cardoso, Renato P Ribas, Sachin S Sapatnekar, and André I Reis. 2006. Fast disjoint transistor networks from BDDs. In *Proc. SBCCI*. 137–142. doi:10.1145/1150343.1150381
- [8] Reinhard Diestel. 2005. *Graph Theory*. Graduate Texts in Mathematics, Vol. 173. Springer. <https://www.math.uni-hamburg.de/home/diestel/books/graph.theory/preview/Ch4.pdf>
- [9] Federal University of Rio Grande do Sul, Logics Lab. 2012. Catalog of 53 Handmade Optimum Switch Networks. [Online]. https://www.inf.ufrgs.br/logics/docman/53_NSP_Catalog.pdf
- [10] Lang Feng, Rongjian Liang, and Hongxin Kong. 2025. Hybrid exact and heuristic efficient transistor network optimization for multi-output logic. In *Proc. DATE*. 1–7. doi:10.23919/DATE64628.2025.10992735
- [11] Nils Froleyks, Marijn Heule, Ashlin Iser, Matti Järvisalo, and Martin Suda. 2021. SAT Competition 2020. *Artificial Intelligence* 301 (2021), 103572. doi:10.1016/j.artint.2021.103572
- [12] Dimitri Kagaris. 2015. MOTO-X: A multiple-output transistor-level synthesis CAD tool. *IEEE Trans. CAD* 35, 1 (2015), 114–127. doi:10.1109/TCAD.2015.2448675
- [13] Dimitri Kagaris and Themistoklis Haniotakis. 2007. A methodology for transistor-efficient supergate design. *IEEE Trans. VLSI* 15, 4 (2007), 488–492. doi:10.1109/TVLSI.2007.895248
- [14] Byeonggon Kang, Alan Mishchenko, Masahiro Fujita, Bill Lin, and Chung-Kuan Cheng. 2026. L2L: Logic to Layout Exploration of Standard Cell Library Design. In *Proc. DAC*. doi:10.1145/3770743.3804220
- [15] Mayler GA Martins, Vinicius Callegaro, Renato P Ribas, and Andre I Reis. 2011. Efficient method to compute minimum decision chains of Boolean functions. In *Proc. GLSVLSI*. 419–422. doi:10.1145/1973009.1973098
- [16] Mayler GA Martins, Leomar Rosa, Anders B Rasmussen, Renato P Ribas, and Andre I Reis. 2010. Boolean factoring with multi-objective goals. In *Proc. ICCD*. 229–234. doi:10.1109/ICCD.2010.5647772
- [17] Vinicius Neves Possani, Vinicius Callegaro, André I Reis, Renato P Ribas, Felipe de Souza Marques, and Leomar Soares Da Rosa. 2015. Graph-based transistor network generation method for supergate design. *IEEE Trans. VLSI* 24, 2 (2015), 692–705. doi:10.1109/TVLSI.2015.2410764
- [18] Vinicius N Possani, Renato S de Souza, Julio S Domingues Jr, Luciano V Agostini, Felipe S Marques, and Leomar S da Rosa Jr. 2012. Optimizing transistor networks using a graph-based technique. *Analog Integr. Circuits Signal Process.* 73, 3 (2012), 841–850. doi:10.1007/s10470-012-9874-z
- [19] Vinicius N Possani, Éric F Timm, Luciano V Agostini, and Leomar S da Rosa. 2011. A graph-based technique to optimize transistor networks. In *Proc. IEEE LASCAS*. 1–4. doi:10.1109/LASCAS.2011.5750285
- [20] Vivek Tiwari, Deo Singh, Suresh Rajgopal, Gaurav Mehta, Rakesh Patel, and Franklin Baez. 1998. Reducing power in high-performance microprocessors. In *Proc. DAC*. 732–737. doi:10.1145/277044.277227
- [21] Jiun-Cheng Tsai, Wei-Min Hsu, Kuei-Lin Wu, Hsuan-Ming Huang, Jen-Hang Yang, Heng-Liang Huang, Yen-Ju Su, and Charles H-P Wen. 2025. MuSTNet: SAT-based Exact Multi-Stage Transistor Network Synthesis with Placement Awareness. In *Proc. ICCAD*. 1–8. doi:10.1109/ICCAD66269.2025.11240651
- [22] Weihua Xiao, Shanshan Han, Yue Yang, Shaoze Yang, Cheng Zheng, Jingsong Chen, Tingyuan Liang, Lei Li, and Weikang Qian. 2023. Minitntk: An exact synthesis-based method for minimizing transistor network. In *Proc. ICCAD*. 01–09. doi:10.1109/ICCAD57390.2023.10323691